#7

CS 0447
Introduction to
Computer Programming

Functions and the Stack

Luís Oliveira

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

Fall 2020

What do I need to know now!

<u>The classes will be recorded!</u>

- You will be able to access the videos online
  - They are for your personal use only!
  - Do not distribute them!

- You don't need to turn on your camera
  - If you do, you may be recorded

- You can ask questions via text!
  - Chat is great for that. If I don't stop and read your questions, ask them again
  - But feel free to interrupt me at any point.

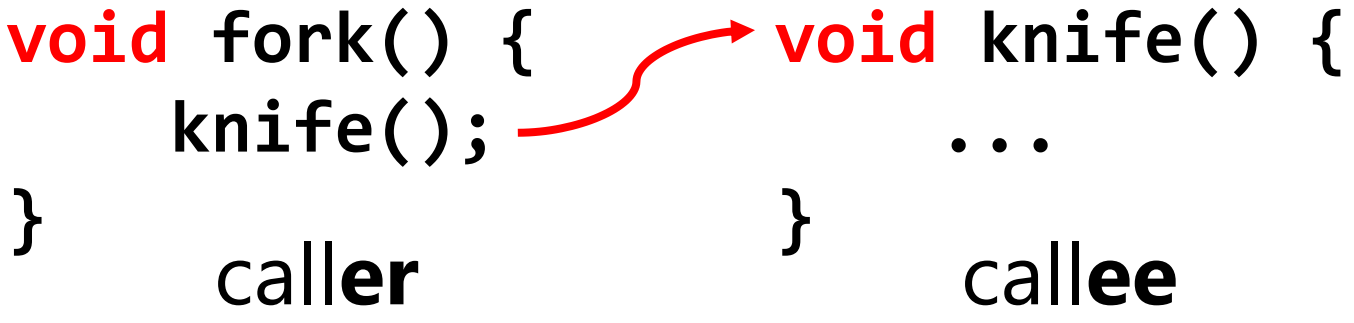
Class announcements

- Project 1 is out soon!
  - You have 3 weeks!
- Don't forget!
  - In the website (next to the slides) there are code examples
  - Those code examples have extra exercises!
  - Go and do them!
- Have you been going to recitation?
  - You should!

Calling conventions

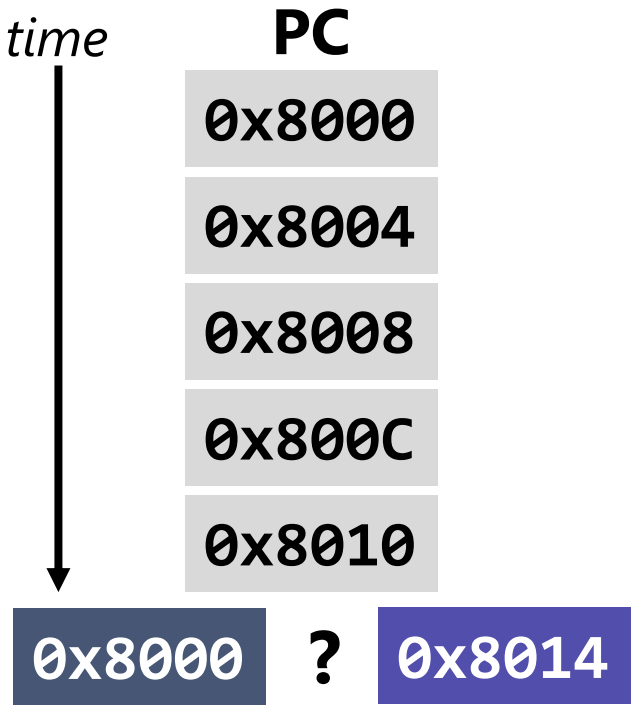
What's a calling convention?

- It ensures our programs don't trip over their own feet
- It's **how machine-language functions call one another**
  - How **arguments** are passed
  - How **return values** are returned
  - How control **flows into/out of the function**
  - What **contracts** exist between the call**er** and the call**ee**



The program counter register

- A program's instructions are in memory, so they have **addresses**
- The PC (program counter) holds the address of **the next instruction to run**
  - **Is incremented by a word!** Each instruction is a word




```
        Address
top:    0x8000  lw   t0, (s0)
        0x8004  add  t0, t0, 1
        0x8008  sw   t0, (s0)
        0x800C  add  s0, s0, 4
        0x8010  blt  s0, s1, top
```

btw what pattern do you notice about these addresses?

MIPS ISA: conditional branch instructions

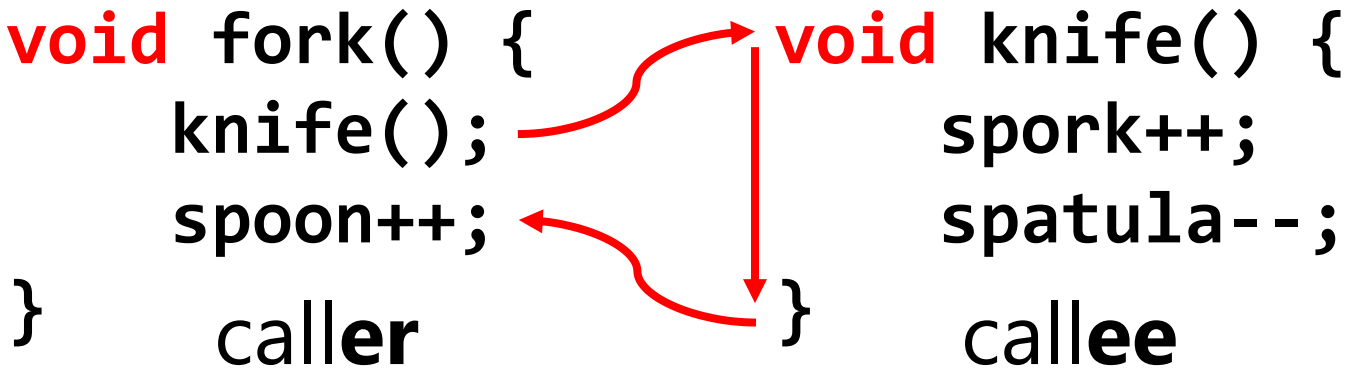
- The **conditional** branch instructions we've seen last class
  - Actually interact with pc

Instruction	Meaning
`beq a, b, label`	`if(a == b) { goto label }`
`bne a, b, label`	`if(a != b) { goto label }`
`blt a, b, label`	`if(a < b) { goto label }`
`ble a, b, label`	`if(a <= b) { goto label }`
`bgt a, b, label`	`if(a > b) { goto label }`
`bge a, b, label`	`if(a >= b) { goto label }`

$pc = label

The flow of control

- When the caller calls a function, where do we go?
- When the callee's code is finished, where do we go?



MIPS ISA: The jump and link instruction

- We call functions with **jal: j**ump **a**nd **l**ink

```
void main() {
  func();
}
```

```
main:
  jal func
```

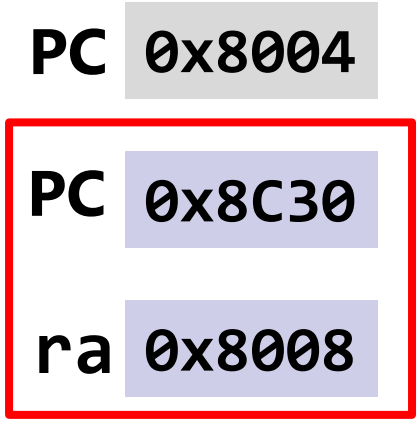
label

MIPS ISA: The jump and link instruction

- We call functions with **jal: j**ump **a**nd **l**ink

What address should go into PC next?

When **func** returns, where will we go?




```
        0x8000  li   a0, 10
        0x8004  jal  func
        0x8008  li   v0, 10
           ...  ...
func:   0x8C30  li   v0, 4
           ...  ...
```

This is what **jal** does:
it **jumps** to a new location, and
makes a **link** back to the old one
in the **ra** (return address) register

and this is ALL it does.

MIPS ISA: The jump register instruction

- We return from functions with **jr**: **j**ump to address in **r**egister

```
void main() {
  func();
}

void func() {
  return;
}
```

```
main:
    jal func
```

label

```
label
func:
    jr ra
```

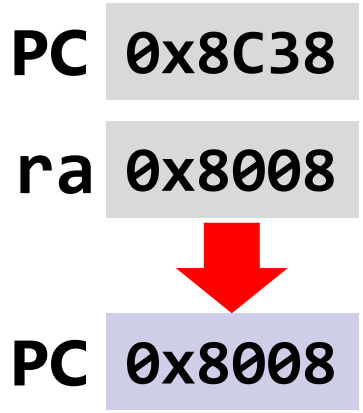
return

MIPS ISA: The jump register instruction

- We return from functions with **jr: j**ump to address in **r**egister

Now we're at the end of **func.** **ra** still has the proper return address

`jr ra` copies **ra** into **pc.** **and this is ALL it does.**




```
          0x8000  li   a0, 10
          0x8004  jal  func
          0x8008  li   v0, 10
             ...  ...

func:     0x8C30  li   v0, 4
          0x8C34  syscall
          0x8C38  jr   ra
```

Arguments and Return Values

It's pretty simple, remember register names!!

- if we have a function in a higher level language…

```
  v0          a0      a1
int gcd(int a, int b) {
   while(a != b) {
       if(a > b)
           a -= b;
       else
           b -= a;
   }
   return a;
}
```

we use particular registers to pass arguments and return values.

we already know how to return. How do syscalls do it?

for this, just *put the value you want to return in **v0** before jr ra.*

The a and v registers

- **a0-a3** are the **argument registers**
- **v0-v1** are the **return value registers**
  - This is just a **convention,** there's nothing special about them
    - Does this mean I can pass values in (e.g.) s-registers?
      - Yesssssss..????¬.¬
    - Will I lose any points in midterms/labs/projects if I do?
      - Yesssssssss!!!!¬.¬



- By convention! We **never** do that!
  - **ALWAYS** pass arguments in ***a-registers***
  - **ALWAYS** return arguments in ***v-registers***

To call a function…

- You **put its arguments in the a registers** before doing a `jal`
- Once control is inside the callee…
  - The arguments are just "there" in the a registers.
    - Cause they are.
      - They didn't go anywhere!

- Functions should be **black boxes** for the caller
  - You don’t need any information about the implementation
  - You only need to know inputs and outputs!
  - … and conventions

`add_nums`

Amazing function that adds two numbers. You do not need to know how it is implemented!!!

Inputs:
1. Number to add
2. Number to add

Outputs:
1. Numbers added together

Let's call a function!

- Let's make **main** do this:

```
v0=add_nums(3, 8)
print(v0)
```

- How do we set 3 and 8 as the arguments?
- How do we call **add_nums**?
- Afterwards, which register holds the sum?
- So how can we print that value?
- *Why* do syscalls put the number of the syscall in **v0**?
  - Well what do you get when you cross an elephant and a rhino?
    - Hell if I know ¯_(ツ)_/¯

```
li   a0, 3
li   a1, 8
jal  add_nums
move a0, v0
li   v0, 1
syscall
```

Careful!

Input, output

- Now, let's write the function:

```
int add_nums(int x, int y) {return x + y;}
```

- inside of our **add_nums** asm function…
  - which register represents **x**?
  - which register represents **y**?
  - which register will hold the sum that we return?

```
add_nums:
    add __, __, __
        v0  a0  a1
    jr  ra
```

More conventions: Saved and Unsaved registers

Let's try something

- Let's make a variable and a function to change it

```
.data
    counter: .word 0
.text
increment:
    la  t0, counter
    lw  t1, (t0)
    add t1, t1, 1
    sw  t1, (t0)
    jr  ra
```

then we can call it

```
main:
    jal increment
    jal increment
    jal increment
```

Everything's just fine, right?

- let's write a loop that calls it ten times in a row
- so we need a loop counter ('i' in a for loop)

```
    li t0, 0 # our counter
loop_begin:
    jal increment
    add t0, t0, 1
    blt t0, 10, loop_begin
loop_end:
```

(this is a do-while loop)

if we run this, it only increments the variable **once.**

why? let's put a **breakpoint** on blt and see what it sees.

Scribbling on someone else's notes

- **both functions are trying to use t0 for different purposes**
  - but there's only ONE **t0**!
- the **increment** function is in the clear
  - the problem is *actually* the loop
- this is one of the **contracts between the caller and the callee…**

a caller cannot depend on the t, a, or v registers to have the same values *after* a call as *before* it.

or to put it another way, callees are allowed to *trash* those registers.

Another piece of the calling convention puzzle

- When you call a function, **it's allowed to change some registers**
- But other registers **must be left exactly as they were**

functions are required to put these registers **back the way they were before they were called.**

Saved	Unsaved
`s0-s7`	`v0-v1`
`sp`	`a0-a3`
`ra*`	`t0-t9`

anyone can change these. after you call a function, **they might have totally different values from before you called it.**

*ra is a little weird cause it's kinda "out of sync" with the other saved regs but you DO save and restore it like the others

Whenever you call a function...

- after a **jal**, **you have no idea what's in these registers.**

```
...
jal  increment
...
```

Unsaved
v0-v1
a0-a3
t0-t9

could be nonsense!
garbage! bogus!

Remember functions are blackbox...
EVEN if YOU implemented it!

Why it broke

- if we look at this code again...

```
    li t0, 0
loop_begin:
    jal increment
    add t0, t0, 1
    blt t0, 10, loop_begin
loop_end:
```

t0 is our loop counter and everything's fiiiine.

uh oh.

WHAT IS IN t0 NOW??

instead, this is a great place to use an **s register.**

Using the convention

- if we use an s register...

```
    li s0, 0
loop_begin:
    jal increment
    add s0, s0, 1
    blt s0, 10, loop_begin
loop_end:
```

s0 is our loop counter and everything's fiiiine.

uh oh.

oh whew, we used an s register, it's fine.

but s registers aren't magic. **they don't do this automatically.**

Don't step on each others' toes

- let's track **PC** and **ra** as we run this code.

	PC	ra
	0x8000	0x0000
After `jal fork`:	0x8020	0x8004
After `jal spoon`:	0x8040	0x8024
After `jr ra`:	0x8024	0x8024
After `jr ra`:	0x8024	0x8024
After `jr ra`:	0x8024	0x8024
After `jr ra`:	0x8024	0x8024
After `jr ra`:	0x8024	0x8024
After `jr ra`:	0x8024	0x8024

UHHHHHHHH!

```
        0x8000  jal fork
        0x8004  li  v0, 10
           ...  ...
fork:   0x8020  jal spoon
        0x8024  jr  ra
           ...  ...
spoon:  0x8040  jr ra
```

What's the deal?

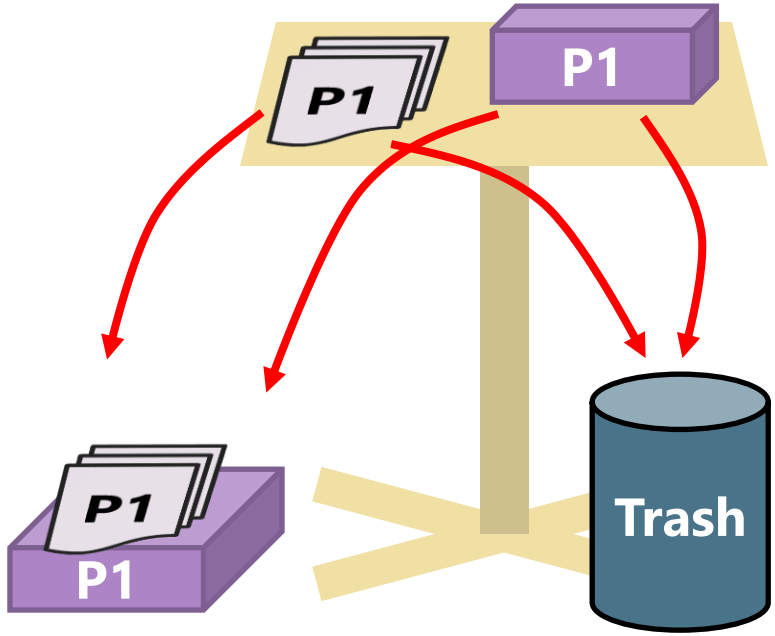
- There's only **one return address register**
- If we call more than **one level deep,** things go horribly wrong
- Could we put it in another register?
  - Then what about three levels deep? four?
    - We just don't have ***enough registers…***
- So **where** do we put things when we don't have room in registers?
  - Tip: NOT in other registers (obviously!)
    - So don’t give into the urge of doing it
  - Put things in memory!

(yes, memory)

The Stack

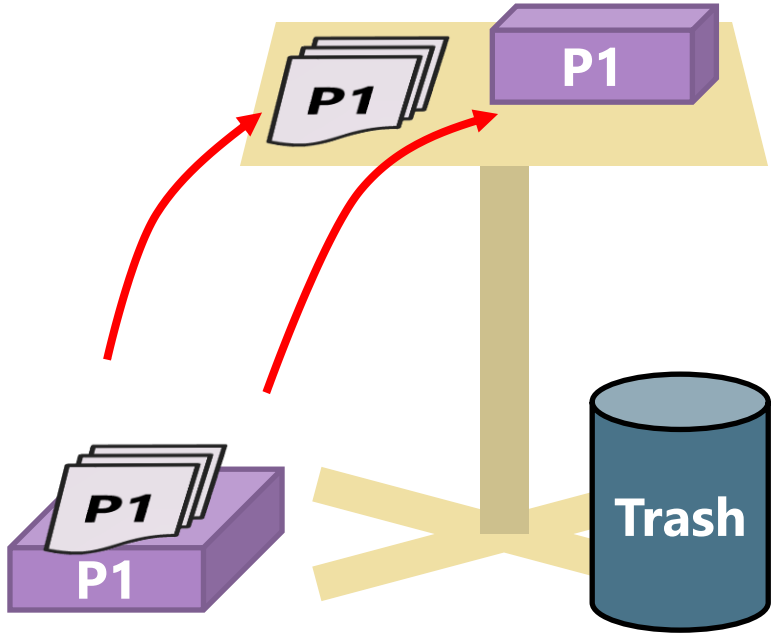
One busy desk

- there's a tiny desk that three people have to share
- person 1 is working at the desk. it's covered in their stuff.
- person 2 **interrupts them** and needs to do some important work
- **what does person 2 do with the stuff?**
  - throw it in the trash?
- they **put it somewhere else.**



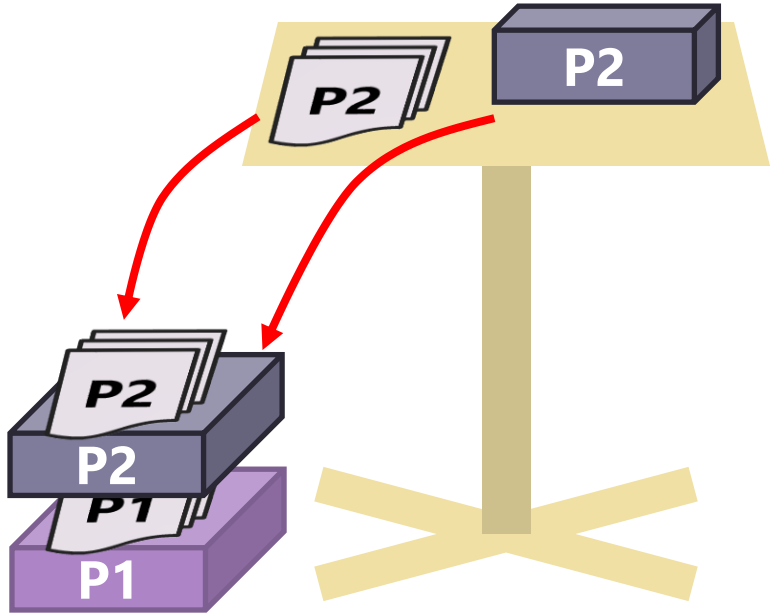
One busy desk

- there's a tiny desk that three people have to share
- person 1 is working at the desk. it's covered in their stuff.
- person 2 **interrupts them** and needs to do some important work
- **what does person 2 do with the stuff?**
  - throw it in the trash?
- they **put it somewhere else.**
- And once they are done
  - They **put it back.**



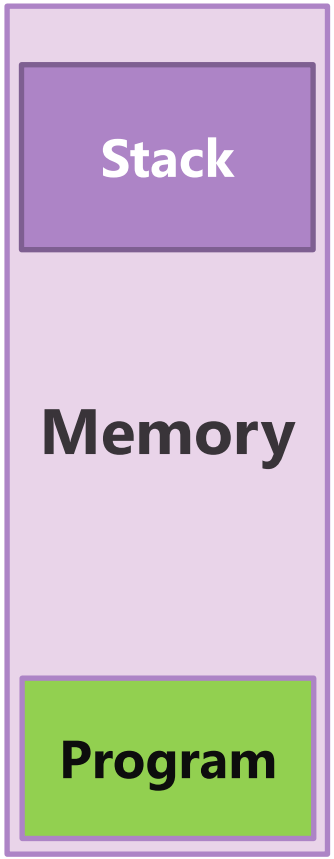
One busy desk

- now person 2 is interrupted by person 3.
- when person 3 is done, person 2 will come back.
- where do we put person 2's stuff?
  - **on top of the stack of stuff.**
- the desk is the **registers.**
- the people are **functions.**
- the stack of stuff is… **the stack.**



What's the stack?

- it's an **area of memory** provided to your program by the OS
  - when your program starts, it's already there
- the stack holds **information about function calls:**
  - **the return address** to the caller
  - **copies of registers** that we want to change
  - **local variables** that can't fit in registers
- how do we access the stack?
  - through the **stack pointer (sp) register**
  - this register is initialized for you by the OS too



The stack pointer (animated)

- let's say **sp** starts at the address **0xF000**
- we want to **push** something on the stack
- the first thing we'll do is **move sp to the next available slot**
- clearly, that's the *previous* address
  - **subtract** 4 from **sp**
- then, we can store something in **the memory that sp points to.**





Doing that in MIPS (animated)

- say **`ra = 0xC0DEBEEF`**
- first: move the stack pointer down (up?):

```
sub sp, sp, 4
```

- then, store **`ra`** at the address that **`sp`** holds.

```
sw  ra, (sp)
```

- now the value in **`ra`** is **saved** on the stack, and **we can get it back later.**
  - and we can store as many return addresses as we want!



Going the other direction (animated)

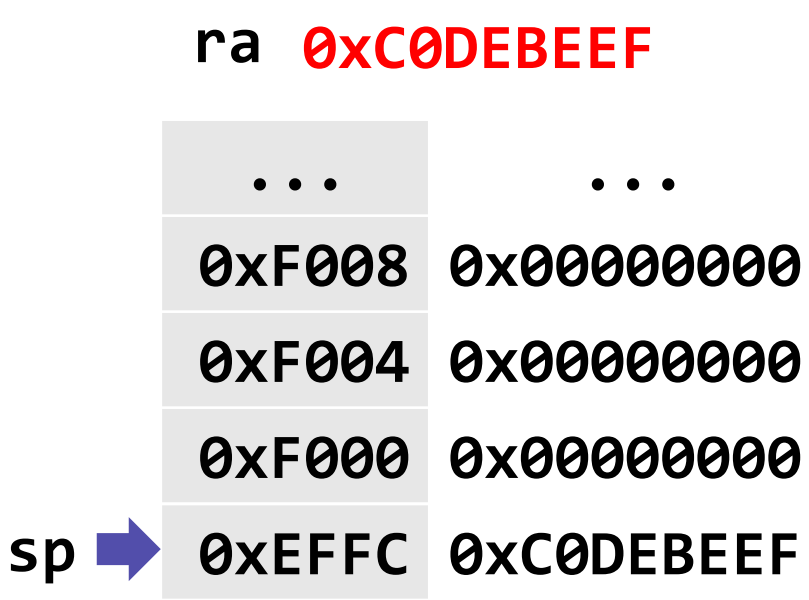
- now we wanna **pop** the value off the stack and put it back in **ra**
- we do the same things, but in reverse

```
lw  ra, (sp)
```

- then, we move the stack pointer…
  up? down? whatever

```
add sp, sp, 4
```

- now we got back the old value of **ra**!
- and **sp** is back where it was before!



Shortening the pushes and pops

- the push and pop operations always look and work the same
- since you'll be using them in most functions, we shortened em
- if you write **push ra** or **pop ra**, it'll do these things for you!

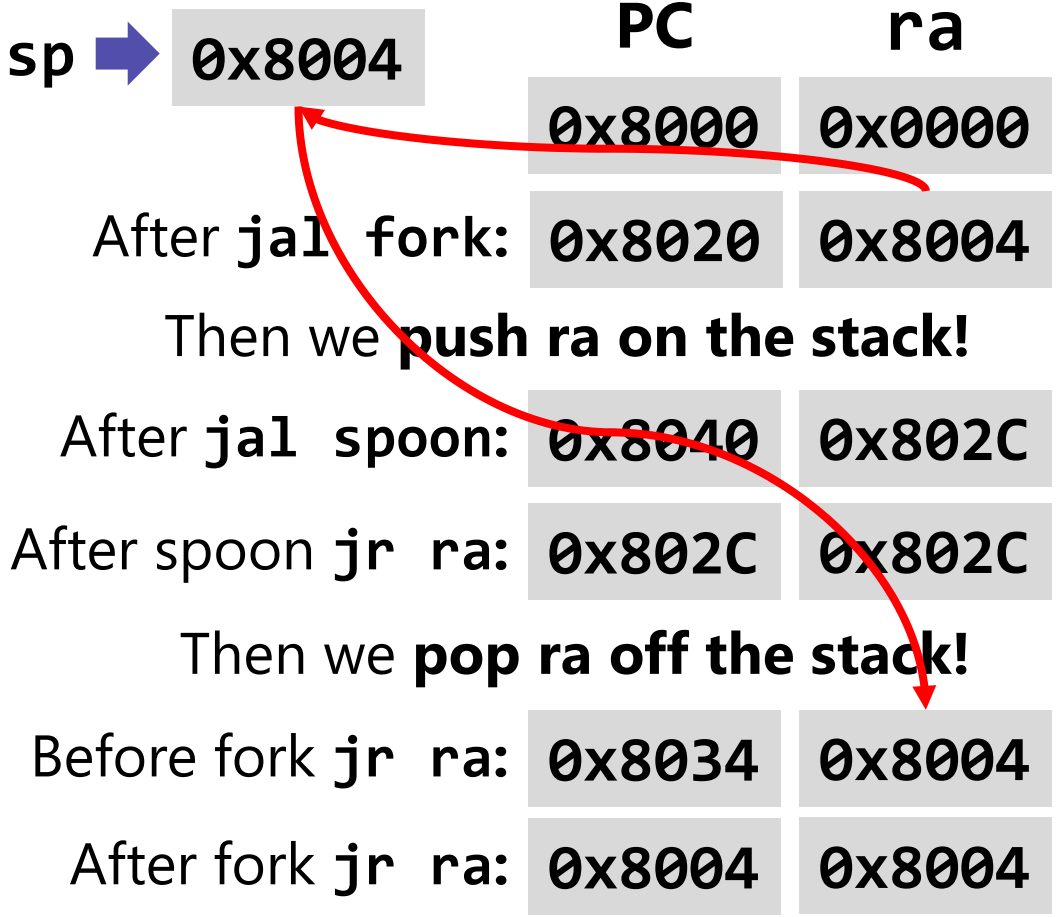
```
push ra  ==  subi sp, sp, 4
             sw   ra, (sp)

pop  ra  ==  lw   ra, (sp)
             addi sp, sp, 4
```

these are **pseudo-ops:** fake instructions to shorten common tasks

these can be used with ANY register, not just **ra!**

Toes = protected




```
0x8000  jal   fork
fork:
0x8020  push  ra
0x8028  jal   spoon
0x802C  pop   ra
0x8034  jr    ra
spoon:
0x8040  jr    ra
```

Writing a simple function

- Function calling conventions follows a simple structure :

1. Give it a name (label). `spoon:`
2. Save **ra** to the stack. `push ra`
3. Do whatever. `your code goes here`
4. Load ra from the stack. `pop ra`
5. Return! `jr ra`

- Push **everything** you need! Pop it back in **reverse order** at the end!

What about other registers?

- Function calling conventions follows a simple structure :

1. Give it a name (label). `spoon:`

2a. Save **ra** to the stack. `push ra`

2b. Save **s0** to the stack. `push s0`

3. Do whatever. `your code goes here`

4a. Load s0 from the stack. `pop s0`

4b. Load ra from the stack. `pop ra`

5. Return! `jr ra`

- Push **everything** you need! Pop it back in **reverse order** at the end!

it's really simple

- treat pushes and pops like the **{ braces }** around a function

```
spoon:
    push ra # {
    # 800 instructions
    # so much stuff omg
    pop  ra # }
    jr   ra
```

pushes come at the beginnings of functions

pops come at the end

that is it, seriously, don't make it more complicated

never push or pop anywhere else please

The s register contract

- if you want to use an s register…
- you must **save and restore it,** just like **ra.**

```
my_func:
    push ra
    push s0
    # code that uses s0! it's fine! we saved it!
    pop  s0
    pop  ra
    jr   ra
```

moving the papers off the desk

putting the papers back

the pops happen in **reverse order!**

Oh, and…

- You must **always** pop the same number of registers that you push.

- To make this simpler for yourself… **make a label before the pops.**
  - then you can leave the function by jumping/branching there.

- Remember: These are the **{ braces }**
  - So… only **push in the top** and **pop in the bottom** of the function!
    - Only!

```
my_func:
    push ra
    push s0
    ...
    bge ...
    b exit_func
    ...
exit_func:
    pop  s0
    pop  ra
    jr   ra
```

Summing it up: Terminology

myFunction:

```
push ra
push s0

# my code

pop  s0
pop  ra
jr        ra
```

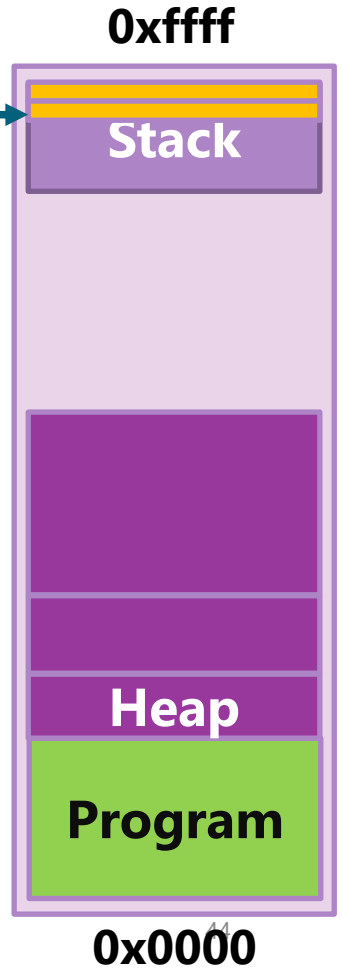
Activation Frame

Contains:

- Arguments (that aren't in registers)
- Saved Registers (ra, s0, etc)
- Local Variables

Function Prologue

Function Epilogue



Side Stacking
(on your own)

Sooooo…..

- Why this mysterious behavior?
  - “Allocating” on the stack (making room) has you *subtract* from its base address.

- Let’s visit this from a different direction.
- Let’s consider… the problem itself.
  - And how we might solve it.



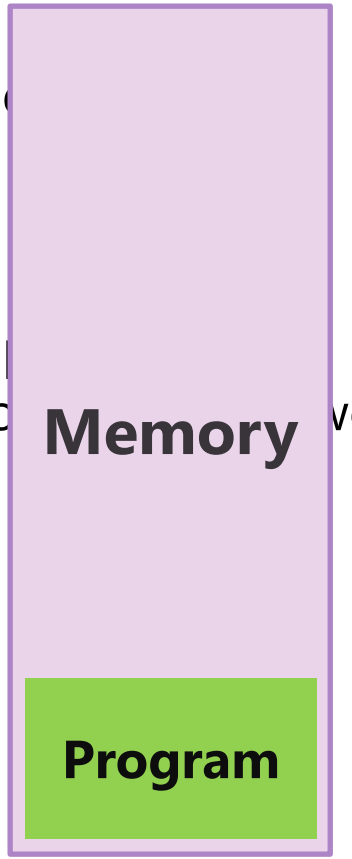
The Problem

- We have a program. It uses memory.
- We don't know exactly how much memory we need.
  - It may depend on **how long the program runs**.
  - Or the **size of the data** it is working on (arbitrarily specified by a human being, perhaps)
  - Maybe our program **responds to the available memory** by choosing a different algorithm when it has more or less.

- Either way, a program does not have a static allocation of memory.
- How do we allow a program to ***allocate*** memory on-demand?

Our Example: Video Editor

- Let's consider a video editing program.
  - But thankfully ignore all of the actual video details!
- Data is large, and the memory usage is relative to the size of our vi
- We want memory to be continuous.
  - Could you imagine if data were all broken up?
  - Your program would be difficult to code if an array was broken u
    - Our array addressing math would no longer be general and wo work well. (You'd have multiple array base addresses)

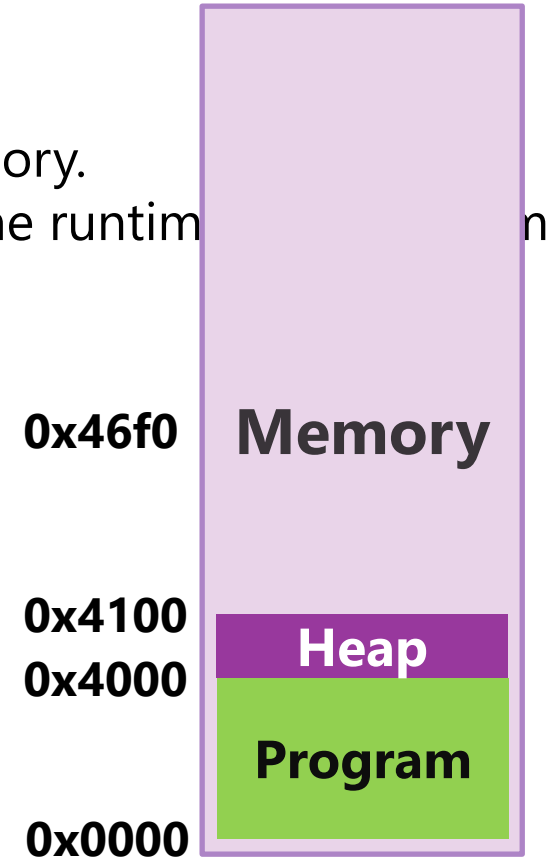


Allocating Memory

- You'll learn a lot more about this in CS 449
  - But it's worth sequence breaking and talking about it now

- We will maintain a section of memory: the **heap**.
  - The heap is a section of memory used for dynamic memory.
  - **Dynamic memory** is memory that is allocated during the runtim n and may be reclaimed later.

- When we allocate memory, we add it to **the end** of the heap.
  - It's like appending to an array.
  - Look at it go!



Revisiting Functions: A Problem Arises

- Now, consider functions.

- When we call a function, we need to remember where we were.
  - This is stored in the $**ra** register.
  - But if we call a function twice, what happens to $**ra**?
    - It is overwritten, and our first value in $*ra* is lost.
    - This means after our second function is called, the first function will now be lost, and it will return *to itself*. (Refer to the previous slides)

- What are our strategies for remembering **ra**?

Remembering RA

- Bad Idea #1: **Place it in another register**

```
myFunction:
    move t0, ra

    # overwrites ra!
    jal myOtherFunction

    # it's ok though:
    move ra, t0
    jr ra
```

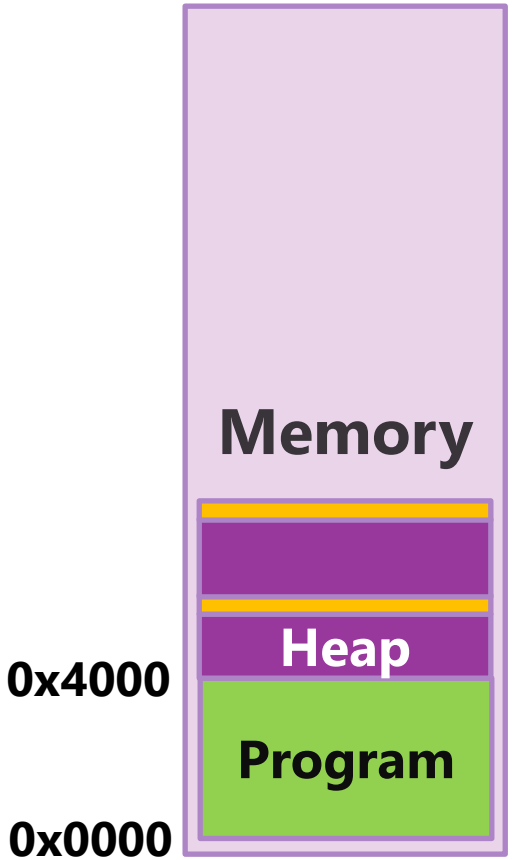
However:

- What if myOtherFunction uses **t0**?
- Ok, t0 isn't preserved, so let's use **s0**.
- Wait… ***we*** need to preserve **s0**…
- Where do we put that?? **s1???**
- **Wait… we need to preserve s1!!**

- We will run out of saved registers and we cannot trust unsaved registers. (other functions may overwrite them)
- Therefore, we need memory.

Remembering RA

- We need memory. We have that *heap thing*.
- So can't we just **allocate some on the heap**?
- Sure can. But it is *Bad Idea #2*.
- What happens if that function allocates memory?
- And then calls another function.
- And then we return…
- And return from the first function…
- Leaving gaps in our memory!



Let's Design a Memory Layout (kinda)

- Our video editing application wants to use large, continuous memory regions.
  - Videos are big things! (Continuous memory makes things easier/faster… future courses will convince you.)

- We have very few registers, and need to remember **ra**
  - So, we need to place **ra** in memory to recall it before we **jr ra**
- However, placing it with other program memory creates gaps
  - This is very very trash!!

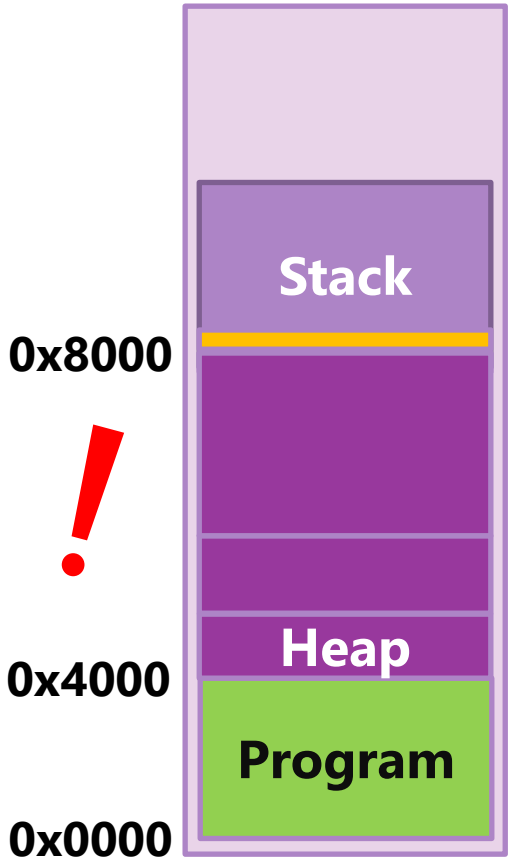
- How do we solve this.
  - Occam's Razor to the rescue… and it will create a very weird situation.
  - One that involves subtracting to allocate…

Solving our Problem: Step 1

- How can we use memory, but not create gaps?

- Good [rational] Idea: Maintain two dynamic memory sections.
- We call our function.
- What happens if that function allocates memory?
- And then calls another function.
- And then we return…
- And return from the first function… WHEW! No gaps.

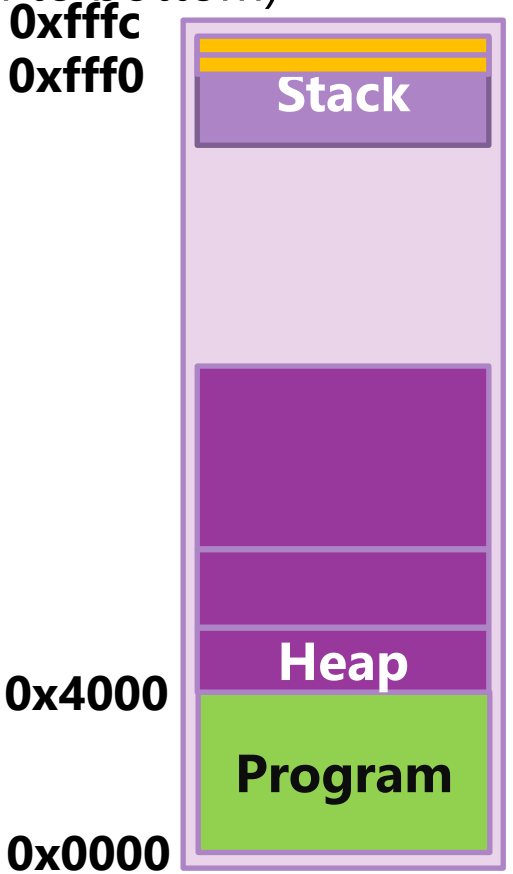
- (Ok, but now we start editing a LARGE video…)
  - Uh oh! We've lost our $**ra**



Solving our Problem: Step 2

- Good [weird] Idea: Maintain two dynamic memory sections. One of which starts at the **highest** memory address. Allocate via **subtraction** (append to bottom)
- We call our function.
- What happens if that function allocates memory?
- And then calls another function.
- And then we return…
- And return from the first function… No gaps.

- As for our large memory case…
- It's fine! (only problem: running out of memory)
  - But, my goodness, you have a bigger problem, then.



Solving our Problem: Step 2

- Good [weird] Idea: Maintain two dynamic memory sections. One of which starts at the **highest** memory address. Allocate via **subtraction** (append to bottom)
- We call our function. **(*subtract* $sp, store)**
- What happens if that function allocates memory?
- And then calls another function. (**sub, store)**
- And then we return… **(load, *add* to $sp)**
- And return from the first function… **(load, *add* $sp)**

- Refer to the previous slides on the Stack with this knowledge in your mind.

